

Custom PyTorch C++/CUDA Operators: Placement and Interview Guide

This guide explains the full path from native code to a PyTorch operator that works on CPU and CUDA, participates in dispatch, and supports gradient-based training. The running example is:

```
mymuladd(a, b, c) = a * b + c
```

Here, **a** and **b** are tensors and **c** is a scalar. The operation is intentionally simple: the focus is the integration architecture, not the arithmetic.

Version note: PyTorch extension APIs evolve. The concepts—schema, dispatch keys, kernels, fake/meta behavior, and derivative formulas—are stable, but always check the documentation for the installed PyTorch version.

C++ Extensions

1. Overview

A **PyTorch C++ extension** is compiled native code that Python or PyTorch can call. It lets a project implement performance-sensitive logic in C++, reuse an existing C/C++ library, or expose an operation that PyTorch does not provide.

A C++ extension is a packaging and integration mechanism. It is not automatically a custom operator: an extension may expose an ordinary Python function through pybind11, register an operator with the PyTorch dispatcher, or do both.

It matters because Python is excellent for model orchestration but is not always the right place for low-level loops, third-party native APIs, or CPU vectorized code. Real systems use extensions for quantization kernels, sparse operations, image/video codecs, graph operations, and inference runtimes. Interviewers ask about them to test whether a candidate understands language boundaries, compilation, tensor metadata, memory ownership, and the difference between merely calling native code and integrating with the PyTorch operator ecosystem.

2. Core Idea

Think of Python as a restaurant waiter and C++ as the kitchen. Python describes the request using tensors; the binding or dispatcher carries that request into native code; C++ performs the work; and the result returns as another tensor. The handoff should avoid copying tensor storage unless the algorithm actually needs a copy.

```
Python call  
  |  
  v
```

```

Binding or dispatcher registration
|
v
C++ function using ATen tensors
|
v
Tensor result sharing PyTorch-managed storage

```

A small CPU implementation using ATen, PyTorch's C++ tensor library, can look like this:

```

#include <torch/extension.h>

at::Tensor mymuladd_cpu(
    const at::Tensor& a,
    const at::Tensor& b,
    double c) {
    TORCH_CHECK(a.device().is_cpu(), "a must be a CPU tensor");
    TORCH_CHECK(b.device().is_cpu(), "b must be a CPU tensor");
    TORCH_CHECK(a.sizes() == b.sizes(), "a and b must have equal shapes");
    TORCH_CHECK(a.scalar_type() == b.scalar_type(), "dtypes must match");
    return a * b + c;
}

```

Step by step:

1. Python creates tensors whose storage is owned by PyTorch.
2. A binding or registered operator receives lightweight `at::Tensor` handles.
3. The C++ code validates the contract at the trust boundary.
4. ATen operations execute on tensor data and return a PyTorch tensor.
5. Reference counting keeps storage alive while tensor handles use it.
6. If this function is registered as an operator, the dispatcher can select it for CPU inputs.

The example composes existing ATen operations, so it is not faster than Python expression `a * b + c`. It is useful for learning integration. A production extension becomes valuable when it calls genuinely optimized native code, avoids intermediate work, or provides missing functionality.

3. Important Subtopics

3.1 ATen and `at::Tensor`

ATen is PyTorch's C++ tensor library. `at::Tensor` is a small handle containing access to tensor metadata and managed storage, rather than a raw array copied across the Python/C++ boundary.

- **Why it matters:** ATen preserves device, dtype, shape, strides, and ownership semantics.
- **Example:** `at::matmul(x, w)` invokes a PyTorch tensor operation from C++.
- **Interview angle:** Explain that passing `at::Tensor` is normally zero-copy; an explicit `.contiguous()`, `.clone()`, or device transfer may allocate and copy.

3.2 pybind11 bindings versus dispatcher registration

pybind11 can expose a C++ function as a Python module function. Dispatcher registration exposes a named operator such as `torch.ops.mylib.mymuladd` to PyTorch subsystems.

```
// Direct Python binding
PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
    m.def("mymuladd", &mymuladd_cpu);
}

// Dispatcher registration
TORCH_LIBRARY(mylib, m) {
    m.def("mymuladd(Tensor a, Tensor b, float c) -> Tensor");
}

TORCH_LIBRARY_IMPL(mylib, CPU, m) {
    m.impl("mymuladd", &mymuladd_cpu);
}
```

- **Why it matters:** A direct binding is sufficient for a private helper, while an operator needs a schema and registrations to cooperate reliably with dispatch, autograd, export, and compilation.
- **Interview angle:** “C++ extension” describes compiled code; “custom operator” describes an operation known to PyTorch.

3.3 Ahead-of-time and just-in-time builds

`torch.utils.cpp_extension.CppExtension` integrates with `setuptools` for an installable package. `torch.utils.cpp_extension.load` compiles and loads sources at runtime, which is convenient for experiments.

```
# setup.py
from setuptools import setup
from torch.utils.cpp_extension import BuildExtension, CppExtension

setup(
    name="mylib_ops",
    ext_modules=[CppExtension("mylib_ops._C", ["mymuladd.cpp"])],
    cmdclass={"build_ext": BuildExtension},
)
```

- **Why it matters:** Build mode affects deployment, caching, reproducibility, and startup time—not kernel semantics.
- **Interview angle:** Use JIT loading for development; package ahead of time for controlled production builds.

3.4 Tensor metadata: shape, dtype, layout, and strides

A tensor is more than a pointer. A correct extension states which shapes, dtypes, layouts, devices, and stride patterns it supports.

```
TORCH_CHECK(a.layout() == c10::kStrided, "strided tensor required");
TORCH_CHECK(a.is_contiguous(), "contiguous tensor required");
TORCH_CHECK(a.scalar_type() == at::kFloat, "float32 required");
```

- **Why it matters:** Treating a transposed tensor as contiguous reads the wrong elements.
- **Example:** `x.t()` can have non-contiguous strides even though it has a valid rectangular shape.
- **Interview angle:** Either write stride-aware logic, use ATen operations that handle strides, or explicitly make a contiguous copy and account for its cost.

3.5 Ownership and lifetime

PyTorch tensors use reference-counted storage. C++ code should keep an `at::Tensor` alive for as long as asynchronous work or retained pointers need its data.

- **Why it matters:** A raw `data_ptr<T>()` does not independently own storage.
- **Example:** Saving a raw pointer globally after the tensor is destroyed creates a use-after-free risk.
- **Interview angle:** Prefer tensor objects and scoped access; never assume Python variable lifetime alone protects asynchronous native work.

3.6 ABI and toolchain compatibility

An extension must be compatible with the compiler, C++ standard library ABI, PyTorch build, Python, and—when CUDA is involved—the CUDA toolchain and target architectures.

- **Why it matters:** Source compatibility does not imply that a previously compiled binary works with a different PyTorch build.
- **Example:** A wheel compiled against one ABI or CUDA runtime may fail to load with an undefined symbol on another environment.
- **Interview angle:** Pin and test the build matrix; rebuild when compatibility is not guaranteed. Stable-ABI APIs can reduce version coupling when their constraints are acceptable.

3.7 Error handling

Use `TORCH_CHECK(condition, message)` for user-visible precondition failures. It creates a Python exception with PyTorch context.

- **Why it matters:** Assertions may disappear in release builds and abrupt process termination is unacceptable inside a Python application.
- **Example:** Reject mismatched dtypes before calling a typed pointer kernel.
- **Interview angle:** Validate at the public boundary, but avoid repeating the same checks in every internal helper.

4. Real-World Example

Suppose an inference service receives compressed image features from a proprietary C++ SDK. Copying the SDK algorithm into Python is impractical. A C++ extension can accept a CPU tensor, call the SDK, and return a tensor:

```

HTTP request -> Python preprocessing -> at::Tensor
      |
      v
    C++ extension wrapper
      |
      v
  Proprietary native SDK
      |
      v
output Tensor -> PyTorch model

```

The wrapper owns contract validation and translation between PyTorch tensors and the SDK's pointer/shape API. The SDK owns its algorithm. This keeps Python orchestration simple without copying the whole data pipeline into a second representation.

5. Diagrams / Mental Models

Extension layers

```

+-----+
| Python package / model      |
+-----+
| Binding or operator schema  |
+-----+
| C++ wrapper and checks      |
+-----+
| ATen or third-party library |
+-----+
| CPU memory and instructions |
+-----+

```

Copy versus handle transfer

```

Python Tensor ---- passes handle ----> C++ at::Tensor
      |                               |
      +----- same Storage -----+

Python Tensor ---- .contiguous() ----> new Storage (possible copy)
Python Tensor ---- .to("cpu") -----> new device Storage (copy)

```

6. Common Interview Questions

1. What is a PyTorch C++ extension?

- **Answer:** Compiled native code integrated with PyTorch, commonly through ATen, pybind11, or dispatcher registration.

- **Expected:** Mention performance/native-library reuse and tensor interoperability.
- **Common mistake:** Calling every C++ extension a CUDA kernel.

2. Why use C++ when PyTorch already has Python APIs?

- **Answer:** To integrate native libraries, implement missing low-level behavior, reduce Python overhead in suitable paths, or write optimized CPU/CUDA kernels.
- **Expected:** Performance must be measured; merely rewriting composed ATen calls in C++ may not help.
- **Common mistake:** Claiming C++ is automatically faster than vectorized PyTorch.

3. Does passing a tensor from Python to C++ copy its data?

- **Answer:** Normally no. `at::Tensor` is a handle to PyTorch-managed storage.
- **Expected:** Copies can occur after operations such as `.clone()`, `.contiguous()`, or device conversion.
- **Common mistake:** Treating `data_ptr()` as owning the memory.

4. What is the difference between pybind11 and `TORCH_LIBRARY`?

- **Answer:** pybind11 exposes a Python-callable native function; `TORCH_LIBRARY` defines an operator schema understood by PyTorch's dispatcher.
- **Expected:** Dispatcher registration enables device-specific kernels and subsystem integration.
- **Common mistake:** Assuming pybind11 alone supplies dispatch or autograd formulas.

5. What information does `at::Tensor` carry?

- **Answer:** Storage reference, shape, strides, dtype, device, layout, and related metadata.
- **Expected:** Distinguish a tensor from a raw contiguous array.
- **Common mistake:** Checking shape while ignoring strides and dtype.

6. Why check tensor contiguity?

- **Answer:** A raw linear-index kernel usually assumes adjacent logical elements are adjacent in memory; non-contiguous tensors violate that assumption.
- **Expected:** Support arbitrary strides or make a documented copy.
- **Common mistake:** Calling `.contiguous()` silently everywhere without considering allocation cost.

7. How should a C++ extension report invalid input?

- **Answer:** Use `TORCH_CHECK` with a useful message at the operator boundary.
- **Expected:** Validate device, dtype, shape, and other assumptions.
- **Common mistake:** Using `assert` for recoverable user errors.

8. What is JIT extension loading?

- **Answer:** `torch.utils.cpp_extension.load` builds and loads sources when the Python program runs or imports them, with build caching.
- **Expected:** Convenient for development, less controlled for deployment.
- **Common mistake:** Confusing build-time JIT loading with GPU kernel runtime compilation.

9. What can cause an extension to fail at import time?

- **Answer:** ABI mismatch, missing shared libraries, incompatible compiler/runtime, unresolved symbols, or a wrong CUDA architecture build.
- **Expected:** Separate compilation errors from dynamic-loader errors.
- **Common mistake:** Debugging every import failure as a Python path problem.

10. How do you test a C++ extension?

- **Answer:** Compare outputs against a clear PyTorch reference across representative shapes, dtypes, devices, strides, empty inputs, and errors; use `opcheck` if it is registered as an operator.
- **Expected:** Test contract and numerical behavior separately.
- **Common mistake:** Testing only one contiguous float32 tensor.

7. Deep-Dive Questions

1. Why can a C++ implementation using only ATen operations have little speed advantage over Python?

- Python already calls the same optimized ATen kernels. Moving the expression into C++ may remove minor interpreter overhead but does not fuse kernels or memory passes by itself.

2. How would you support non-contiguous tensors?

- Use ATen operations or an iterator abstraction that respects strides, write explicit stride-aware indexing, or create contiguous copies. The right choice depends on frequency, performance, and contract.

3. What happens when two `at::Tensor` objects alias the same storage?

- A mutation through one may be visible through the other. Operator schemas and autograd need correct alias/mutation information so transformations and version checks remain valid.

4. Why are extension binaries coupled to their build environment?

- They link against native symbols and ABIs from PyTorch, the standard library, Python, and possibly CUDA. A source-level API match does not guarantee binary compatibility.

5. When should you release the Python GIL?

- For long native CPU work that does not touch Python objects, releasing it can allow other Python threads to progress. Do not release it while calling Python APIs, and remember that dispatcher integration and PyTorch's own threading are separate concerns.

8. Comparison Tables

C++ extension	Pure Python implementation
Compiled native code	Interpreted Python orchestration
Can call C/C++ libraries directly	Best access to Python ecosystem
Requires a compiler and ABI compatibility	Easier installation and debugging

C++ extension	Pure Python implementation
Can implement low-level kernels	Usually composes existing PyTorch ops
More deployment complexity	Faster iteration
pybind11 function	Dispatcher-registered custom operator
Called as a module function	Called through <code>torch.ops.namespace.op</code> or a wrapper
No operator schema by default	Has a typed operator schema
No automatic device-key selection	Can have CPU, CUDA, Meta, Autograd, and other registrations
Suitable for private native utilities	Suitable for PyTorch ecosystem integration
Ahead-of-time package	JIT load build
Built before deployment	Built on demand
Reproducible release artifact	Convenient experimentation
CI produces wheels/packages	Local toolchain required at first build
Better production startup	Faster edit-build-test loop

9. Common Mistakes

- Rewriting a vectorized ATen expression in C++ and assuming it is now fused.
- Reading tensor data with the wrong C++ type for its `scalar_type()`.
- Ignoring non-contiguous strides.
- Keeping a raw pointer after the owning tensor can die.
- Registering only a pybind11 function and expecting `torch.compile` or dispatch integration.
- Performing CPU work on a CUDA tensor pointer.
- Returning a tensor that violates the declared mutation or aliasing behavior.
- Using `assert` instead of a user-facing runtime check.
- Building on the user's machine without pinning compatible compilers and libraries.
- Benchmarking compilation or first-call initialization as steady-state execution.

10. Edge Cases / Special Cases

- **Zero-sized tensors:** A kernel should produce a correctly shaped empty result and avoid invalid zero-block launches.
- **Scalar tensors:** A zero-dimensional tensor has no ordinary length dimension but still has one element.
- **Non-contiguous views:** Transpose, slicing, and channels-last formats can have valid but unexpected strides.
- **Overlapping storage:** Expanded tensors may have zero strides; writing through such views can be unsafe or ambiguous.
- **Mixed dtypes:** Decide whether to reject them or implement PyTorch-style promotion deliberately.
- **Large tensors:** Index calculations may require 64-bit indexing even when each dimension fits in 32 bits.
- **Threading:** ATen may use internal CPU thread pools; adding another thread pool can oversubscribe cores.

- **Exceptions:** Do not allow arbitrary exceptions to cross an ABI boundary unhandled; PyTorch binding machinery translates supported C++ exceptions.
- **Forked processes:** Native thread pools and accelerator contexts may interact poorly with `fork`; data-loader/process design matters.
- **Build cache:** Stale binaries can obscure source changes; know where JIT builds are cached.

11. How to Explain in Interview

"A PyTorch C++ extension is compiled native code that exchanges tensors with PyTorch through ATen. Passing an `at::Tensor` normally shares PyTorch-managed storage, so I must respect its device, dtype, shape, and strides. For a private helper I can use `pybind11`; for an operation that needs device dispatch, autograd, export, or compilation support, I define a schema and register it with the PyTorch dispatcher."

12. Quick Revision Notes

- `at::Tensor` is a metadata-and-storage handle, not merely `T*`.
- C++ extension does not imply CUDA and does not imply custom operator.
- `pybind11` exposes functions; dispatcher registration defines PyTorch operators.
- Validate device, dtype, shape, layout, strides, and alias assumptions.
- `.contiguous()` may copy.
- C++ composition of existing ops is not automatic fusion.
- `CppExtension/BuildExtension`: packaged build.
- `load`: convenient on-demand build.
- `TORCH_CHECK`: user-visible contract enforcement.
- Interview trap: zero-copy handle transfer does not make every operation zero-copy.

13. Practice Tasks

1. Implement `mymuladd_cpu` with ATen operations and expose it through `pybind11`.
2. Register the same function with `TORCH_LIBRARY` and a CPU implementation key.
3. Compare it with `a * b + c` for scalar, empty, and multidimensional tensors.
4. Pass `a.t()` and determine whether the implementation supports its strides.
5. Add clear errors for mismatched shape, dtype, and device.
6. Inspect `sizes()`, `strides()`, `scalar_type()`, and `device()` in C++.
7. Benchmark the C++ composition and Python reference after warm-up; explain why timings are similar.
8. Build once with `CppExtension` and once with `load`; document deployment differences.

14. Final Cheat Sheet

Item	Must remember
Core definition	Compiled C++ code integrated with PyTorch tensor APIs
Main C++ type	<code>at::Tensor</code>
Direct Python exposure	<code>pybind11</code>
Operator integration	<code>TORCH_LIBRARY</code> + <code>TORCH_LIBRARY_IMPL</code>

Item	Must remember
Key contract	Device, dtype, shape, strides, mutation, ownership
Build choices	Packaged ahead-of-time or on-demand load
Most asked	Copy semantics, contiguity, pybind11 vs dispatcher, ABI
Main trap	C++ does not automatically make ATen operations faster or fused
One-line answer	"A C++ extension lets PyTorch call native code while retaining tensor metadata and managed storage."

CUDA Extensions

1. Overview

A **PyTorch CUDA extension** adds native CUDA code—usually **.cu** kernels plus C++ registration or binding code—to a PyTorch project. It lets a tensor operation run directly on NVIDIA GPU memory while remaining callable from PyTorch.

CUDA extensions matter when a workload needs an operation that PyTorch does not offer, when several memory-bound steps can be fused, or when a domain-specific kernel can exploit data layout better than a generic implementation. They appear in attention kernels, custom quantization, differentiable rendering, sparse/graph processing, point-cloud systems, and scientific computing. Interviewers ask about them because correct GPU integration requires reasoning about thousands of threads, asynchronous streams, tensor layout, launch errors, and device ownership at once.

2. Core Idea

Think of the CPU as a dispatcher at a warehouse and the GPU as a large workforce. The CPU does not multiply every element. It prepares the job, chooses how many GPU workers to launch, submits the work to the current stream, and continues. Each GPU thread computes one or more output elements.

For contiguous, equally shaped tensors, a teaching kernel is:

```
#include <ATen/cuda/CUDAContext.h>
#include <c10/cuda/CUDAGuard.h>
#include <c10/cuda/CUDAException.h>
#include <torch/extension.h>

template <typename scalar_t>
__global__ void mymuladd_kernel(
    const scalar_t* a,
    const scalar_t* b,
    scalar_t* out,
    int64_t n,
    scalar_t c) {
    const int64_t i =
        static_cast<int64_t>(blockIdx.x) * blockDim.x + threadIdx.x;
```

```

    if (i < n) out[i] = a[i] * b[i] + c;
}

at::Tensor mymuladd_cuda(
    const at::Tensor& a,
    const at::Tensor& b,
    double c) {
    TORCH_CHECK(a.is_cuda() && b.is_cuda(), "inputs must be CUDA tensors");
    TORCH_CHECK(a.device() == b.device(), "inputs must be on the same GPU");
    TORCH_CHECK(a.sizes() == b.sizes(), "inputs must have equal shapes");
    TORCH_CHECK(a.scalar_type() == b.scalar_type(), "dtypes must match");
    TORCH_CHECK(a.is_contiguous() && b.is_contiguous(), "inputs must be
contiguous");

    c10::cuda::CUDAGuard device_guard(a.device());
    auto out = at::empty_like(a);
    const int64_t n = a.numel();
    if (n == 0) return out;

    constexpr int threads = 256;
    const int blocks = static_cast<int>((n + threads - 1) / threads);
    auto stream = at::cuda::getCurrentCUDASTream();

    AT_DISPATCH_FLOATING_TYPES(a.scalar_type(), "mymuladd_cuda", [&] {
        mymuladd_kernel<scalar_t><<<blocks, threads, 0, stream>>>(
            a.data_ptr<scalar_t>(),
            b.data_ptr<scalar_t>(),
            out.data_ptr<scalar_t>(),
            n,
            static_cast<scalar_t>(c));
    });
    C10_CUDA_KERNEL_LAUNCH_CHECK();
    return out;
}

TORCH_LIBRARY_IMPL(mylib, CUDA, m) {
    m.impl("mymuladd", &mymuladd_cuda);
}

```

Step by step:

1. The dispatcher selects the CUDA registration because tensor inputs are CUDA tensors.
2. The wrapper validates shape, dtype, device, and contiguity.
3. `CUDAGuard` selects the input tensor's GPU, which matters in multi-GPU processes.
4. `empty_like` allocates output with matching device, dtype, layout, and size.
5. The wrapper gets PyTorch's current CUDA stream instead of inventing a private synchronization domain.
6. `AT_DISPATCH_FLOATING_TYPES` instantiates the typed kernel matching the runtime dtype.
7. A grid of threads processes elements; the bounds check protects the partial final block.
8. The launch check catches configuration/launch failures. Execution is still normally asynchronous to the host.

3. Important Subtopics

3.1 Host wrapper versus device kernel

The C++ host wrapper runs on the CPU and handles tensor validation, allocation, device/stream selection, and kernel launch. The `__global__` function runs on the GPU.

- **Why it matters:** Host code may use rich C++/ATen facilities; device code follows CUDA's execution and language restrictions.
- **Example:** `at::empty_like(a)` is host code; `out[i] = ...` is device code.
- **Interview angle:** A kernel launch is a CPU action that queues GPU work; it is not the work itself.

3.2 Grid, blocks, and threads

CUDA organizes threads into blocks and blocks into a grid. A common one-dimensional mapping is `i = blockIdx.x * blockDim.x + threadIdx.x`.

- **Why it matters:** Launch geometry determines coverage and influences occupancy and memory access.
- **Example:** For 1,000 elements and 256 threads per block, launch 4 blocks; extra threads exit due to `i < n`.
- **Interview angle:** Ceiling division plus a bounds check is the standard answer; never assume `n` is divisible by block size.

3.3 Memory coalescing

Adjacent GPU threads should usually access adjacent addresses, allowing the hardware to combine memory transactions.

- **Why it matters:** Elementwise kernels are commonly limited by memory bandwidth rather than arithmetic.
- **Example:** Thread `i` reading `a[i]` is coalesced for contiguous tensors.
- **Interview angle:** A mathematically correct strided access pattern may be much slower because a warp touches many memory segments.

3.4 Dtype dispatch

Templates compile specialized code for each scalar type, while `AT_DISPATCH_*` selects the correct specialization at runtime.

- **Why it matters:** Interpreting a float pointer as double is incorrect, and one binary operator may support several dtypes.
- **Example:** `AT_DISPATCH_FLOATING_TYPES` normally covers floating types represented by that macro, but not every PyTorch dtype.
- **Interview angle:** State exactly which types are supported; half/bfloat16 often need additional dispatch macros and accumulation decisions.

3.5 CUDA streams

A stream is an ordered queue of GPU operations. A custom kernel should normally launch on PyTorch's **current stream** for the relevant device.

- **Why it matters:** Launching on an unrelated stream can race with producers or consumers unless explicit events connect them.
- **Example:** An earlier PyTorch operation producing `a` and this kernel are correctly ordered when both use the same current stream.
- **Interview angle:** The legacy/default stream is not a safe substitute for respecting the framework's current-stream semantics.

3.6 Asynchronous execution and errors

Kernel launches normally return before GPU execution completes. There are immediate launch errors and delayed execution errors.

- **Why it matters:** Timing and debugging are wrong if the host never waits at a meaningful boundary.
- **Example:** Invalid launch configuration may be reported immediately; an out-of-bounds access may surface at later synchronization.
- **Interview angle:** Check the launch, then use a synchronization boundary during testing to reveal execution failures. Avoid device-wide synchronization in the normal operator path.

3.7 Device guards and multi-GPU correctness

A tensor records its CUDA device. The active device in the host thread may be different.

- **Why it matters:** Allocation or launch on the wrong device can fail or access unrelated memory.
- **Example:** Input on `cuda:1` while the current device is `cuda:0`.
- **Interview angle:** Validate that related inputs share a device and guard the input device around device-specific work.

3.8 Build configuration

`CUDAExtension` compiles C++ sources with the host compiler and `.cu` sources with NVCC, then links them into one extension.

```
from setuptools import setup
from torch.utils.cpp_extension import BuildExtension, CUDAExtension

setup(
    name="mylib_ops",
    ext_modules=[CUDAExtension(
        "mylib_ops._C",
        ["mymuladd.cpp", "mymuladd_cuda.cu"],
    )],
    cmdclass={"build_ext": BuildExtension},
)
```

- **Why it matters:** Host code and device code follow different compilation paths.
- **Interview angle:** CUDA toolkit availability, compute architectures, compiler compatibility, and binary size are deployment concerns.

3.9 Numerical precision

Floating-point operations are not exact, and fused multiply-add may round differently from separate multiply and add operations.

- **Why it matters:** A faster fused kernel can be numerically close but not bit-identical to a reference.
- **Example:** Float16 input may accumulate in float32 for stability, then cast back.
- **Interview angle:** Use tolerance-based checks and choose accumulation types deliberately.

4. Real-World Example

An inference system applies bias, activation, and quantization after a matrix multiplication. Three separate PyTorch operators may read and write large intermediate tensors:

```
Matmul output -> [add bias] -> temp1 -> [activation] -> temp2 -> [quantize] ->
output
```

A fused CUDA extension can do the elementwise post-processing in one kernel:

```
Matmul output -> [bias + activation + quantize in registers] -> output
```

The value is usually reduced global-memory traffic and launch overhead, not fewer mathematical operations. A production implementation must still handle alignment, dtype, vectorization tails, streams, shapes, and numerical behavior.

5. Diagrams / Mental Models

Launch hierarchy

```
Grid
+-----+-----+-----+
| Block 0          | Block 1          | ... |
| t0 t1 t2 ... t255 | t0 t1 t2 ... t255 |   |
+-----+-----+-----+
      |   |   |           |
      v   v   v           v
out[0], out[1], ... out[256], ...
```

Correct stream ordering

```
Current stream: [produce a] -> [custom kernel reads a] -> [consumer reads out]
```

Wrong private stream without event:

```
Stream A:          [produce a] ----->
```

```
Stream B:          [custom kernel reads a]  race!
```

Performance model

Kernel type	Likely bottleneck	First optimization questions
Simple elementwise	Memory bandwidth / launch overhead	Are accesses coalesced? Can operations be fused?
Reduction	Synchronization and memory traffic	Warp reductions? Shared memory? Numerically stable order?
Matrix-like	Compute throughput and data reuse	Tiling? Shared memory? Tensor cores? Library primitive?
Irregular graph/sparse	Divergence and random access	Work balance? Locality? Atomics contention?

6. Common Interview Questions

1. What is a PyTorch CUDA extension?

- **Answer:** Native CUDA kernels plus integration code compiled so PyTorch can call them on CUDA tensors.
- **Expected:** Mention tensor contracts, streams, and registration/bindings.
- **Common mistake:** Describing it only as “C++ code that is faster.”

2. Why does a CUDA kernel need a bounds check?

- **Answer:** The grid is usually rounded up to whole blocks, so the final block may contain threads beyond `numel()`.
- **Expected:** `if (i < n)` after ceiling division.
- **Common mistake:** Launching too few blocks using floor division.

3. Why use PyTorch's current CUDA stream?

- **Answer:** It preserves ordering with operations surrounding the custom op and works with user-selected streams.
- **Expected:** Streams express dependencies; arbitrary streams can race.
- **Common mistake:** Calling `cudaDeviceSynchronize()` to hide a stream bug.

4. Is a kernel launch synchronous?

- **Answer:** Normally it is asynchronous with respect to the host; it is enqueued into a stream.
- **Expected:** Same-stream operations remain ordered.
- **Common mistake:** Equating asynchronous submission with concurrent execution.

5. What does memory coalescing mean?

- **Answer:** Threads in a warp access nearby addresses so hardware can serve them with fewer memory transactions.
- **Expected:** Connect it to contiguous indexing and bandwidth.
- **Common mistake:** Saying coalescing puts data in shared memory.

6. How do you support multiple dtypes?

- **Answer:** Validate the runtime dtype and use ATen dispatch macros/templates to launch a correctly typed specialization.
- **Expected:** State support for half/bfloat16 and accumulator types explicitly.
- **Common mistake:** Casting `data_ptr()` without checking dtype.

7. Why use a device guard?

- **Answer:** To make allocation, stream lookup, and launch occur in the input tensor's device context.
- **Expected:** Multi-GPU correctness.
- **Common mistake:** Assuming the process has only one active GPU.

8. How should CUDA kernel errors be checked?

- **Answer:** Check immediately after launch for launch errors and force/observe a completion boundary during tests for delayed execution errors.
- **Expected:** Do not add a global sync to every production call.
- **Common mistake:** Checking only the C++ function return.

9. When is a custom CUDA kernel actually faster?

- **Answer:** When it reduces launches or memory traffic, exploits domain-specific structure, or supplies an implementation unavailable in optimized libraries.
- **Expected:** Benchmark against PyTorch and vendor primitives after warm-up.
- **Common mistake:** Assuming handwritten CUDA beats cuBLAS/cuDNN.

10. How do you time a CUDA extension correctly?

- **Answer:** Warm up, use CUDA events or synchronize around host timing, repeat enough iterations, and include/exclude allocation consistently.
- **Expected:** Explain asynchronous launch behavior.
- **Common mistake:** Timing only the Python call without synchronization.

7. Deep-Dive Questions

1. Why can using the wrong stream produce nondeterministic results instead of an immediate error?

- Valid device pointers are still valid on both streams, but without an ordering dependency one stream can read before another finishes writing. The race depends on timing.

2. How would you extend the one-thread-per-element kernel beyond the grid dimension limit?

- Use a grid-stride loop: each thread processes `i`, then `i += blockDim.x * gridDim.x`, covering arbitrarily large tensors with a bounded grid.

3. Why might float16 input need float32 accumulation?

- Float16 has limited precision and range. Accumulating partial sums in float32 reduces rounding and overflow risk, though it changes performance and must match the operator's documented

numerical contract.

4. What is occupancy, and why is maximizing it not always the goal?

- Occupancy measures active warps relative to hardware capacity. More active warps can hide latency, but register pressure, shared-memory use, instruction throughput, and memory bandwidth can make a lower-occupancy kernel faster.

5. How do asynchronous allocators affect tensor lifetime?

- Storage reuse must be ordered with the stream that uses it. PyTorch's allocator and stream-recording mechanisms manage this for normal tensor operations; custom code must not detach raw pointer lifetime from tensor and stream lifetime.

8. Comparison Tables

CPU extension kernel	CUDA extension kernel	
Executes on CPU cores	Executes across GPU threads	
Normal C++ function call	Asynchronous kernel launch	
Host memory pointers	Device memory pointers	
CPU threading/vectorization	Blocks, warps, occupancy, coalescing	
No CUDA stream	Must respect current CUDA stream	
Host compiler	Host compiler plus NVCC/device toolchain	
Global memory	Shared memory	Registers
Large, high latency	Per-block, software-managed	Per-thread, fastest
Visible across grid	Shared by threads in a block	Private to a thread
Persists across kernels	Lifetime of block	Lifetime of thread/kernel values
Coalescing matters	Bank conflicts matter	Pressure can reduce occupancy
Synchronization choice	Scope	Typical use
<code>__syncthreads()</code>	Threads in one block	Shared-memory phase boundary
CUDA event/wait	Selected streams	Cross-stream dependency
Stream synchronize	Host waits for one stream	Result needed on CPU/testing
Device synchronize	Host waits for all device work	Debugging or rare global boundary

9. Common Mistakes

- Launching on a hard-coded/default stream instead of PyTorch's current stream.
- Forgetting the input device guard in a multi-GPU process.
- Treating a non-contiguous tensor as a flat contiguous array.
- Launching zero blocks for an empty tensor.

- Using 32-bit indexing for a tensor whose total element count can overflow it.
- Omitting launch error checks.
- Adding `cudaDeviceSynchronize()` inside the operator and destroying asynchrony.
- Assuming more threads or higher occupancy always means higher performance.
- Comparing against a reference without warm-up or synchronization.
- Ignoring float16/bfloat16 accumulation and tolerance behavior.
- Handwriting a slower version of an optimized library primitive.

10. Edge Cases / Special Cases

- **Empty tensors:** Return an allocated empty output before launching.
- **Very large tensors:** Use 64-bit indices or a safe grid-stride scheme.
- **Non-contiguous/channels-last:** Either support strides/layouts or explicitly constrain inputs.
- **Multiple GPUs:** Reject cross-device input combinations unless peer access and semantics are deliberately implemented.
- **Current stream capture:** CUDA Graph capture requires capture-safe allocation and API behavior.
- **Unified/managed memory:** Addressability does not remove synchronization or migration costs.
- **Half and bfloat16:** Intrinsics, vector types, and accumulator choices differ.
- **Alignment:** Vectorized loads require alignment and a scalar tail path.
- **Divergence:** Branch-heavy kernels may serialize paths within a warp.
- **Atomics:** Results may be nondeterministic for floating-point reductions due to update order.
- **Architecture targeting:** A binary may lack code for a newer/older GPU if build architectures were misconfigured.
- **Watchdog/timeouts:** Very long kernels can fail on display GPUs even if logically correct.

11. How to Explain in Interview

"A PyTorch CUDA extension has a CPU-side wrapper and a GPU kernel. The wrapper validates tensor metadata, selects the tensor's device, allocates output, and launches on PyTorch's current CUDA stream. The kernel maps threads to elements with bounds checks and coalesced access. I check launch errors, avoid global synchronization, test edge cases like empty and non-contiguous tensors, and benchmark against native PyTorch after warm-up."

12. Quick Revision Notes

- Wrapper runs on CPU; `__global__` kernel runs on GPU.
- Use the tensor's device and PyTorch's current stream.
- Ceiling-divide blocks and bounds-check the last block.
- Adjacent threads + adjacent elements usually improve coalescing.
- Dtype dispatch connects runtime type to a compiled template.
- Launch is normally host-asynchronous.
- Check immediate launch errors and delayed execution errors.
- Empty tensor: do not launch a zero-sized grid.
- Benchmark with warm-up and proper synchronization/events.
- Interview trap: custom CUDA is not automatically faster than vendor libraries.

13. Practice Tasks

1. Implement the contiguous `mymuladd` CUDA kernel and compare it with `a * b + c`.
2. Add a grid-stride loop and test a tensor larger than one grid's direct coverage.
3. Test empty tensors, scalars, odd sizes, and sizes not divisible by 256.
4. Pass a transposed tensor; reject it clearly, then implement a stride-aware alternative.
5. Add float64 support with dtype dispatch and verify tolerances.
6. Run on `cuda:1` while `cuda:0` is current and verify the device guard.
7. Use two PyTorch streams and events to prove the operator respects current-stream ordering.
8. Benchmark separate multiply/add kernels versus the fused kernel using CUDA events.
9. Intentionally create an out-of-bounds access in a disposable exercise and observe where the asynchronous error appears.

14. Final Cheat Sheet

Item	Must remember
Core definition	Native CUDA code callable as a PyTorch tensor operation
Host duties	Validate, guard device, allocate, select stream, launch, check
Kernel duties	Map threads, bounds-check, compute, write safely
Performance basics	Coalescing, fusion, occupancy, launch overhead, precision
Correct stream	<code>at::cuda::getCurrentCUDAStream()</code> for the guarded device
Multi-GPU safety	Same-device validation + device guard
Most asked	Indexing, streams, asynchrony, coalescing, timing
Main trap	Correct arithmetic can still race when launched on the wrong stream
One-line answer	"A CUDA extension launches a validated, stream-correct GPU kernel over PyTorch-managed device tensors."

Custom Operators

1. Overview

A **custom operator** is a named operation with a formal schema and one or more implementations registered into PyTorch. The schema describes inputs, outputs, types, defaults, mutation, and aliasing. Implementations—also called kernels—provide behavior for CPU, CUDA, Meta/FakeTensor, autograd, autocast, batching, or other dispatch contexts.

The key idea is that a custom operator is not just a Python or C++ function. It creates an opaque semantic boundary that PyTorch systems can identify as `namespace::operator`, such as `mylib::mymuladd`. This matters for `torch.compile`, `torch.export`, FX graphs, autograd, device dispatch, and tensor subclasses.

Custom operators are used to wrap third-party libraries, expose handwritten C++/CUDA kernels, preserve an opaque call in compiled graphs, and introduce domain operations. Interviewers ask about them because the

design tests API contracts, mutation and aliasing, multi-backend implementation, compiler compatibility, and testing discipline.

2. Core Idea

Think of an operator schema as a service contract and kernels as regional service implementations. Clients call one stable service name. The runtime selects an implementation appropriate for the request.

```
Stable identity:  mylib::mymuladd
Schema:          (Tensor a, Tensor b, float c) -> Tensor
Implementations: CPU | CUDA | Meta/Fake | Autograd | Autocast | ...
```

A Python-defined teaching version is:

```
import torch
from torch import Tensor

@torch.library.custom_op("mylib::mymuladd", mutates_args=())
def mymuladd(a: Tensor, b: Tensor, c: float) -> Tensor:
    if a.shape != b.shape:
        raise ValueError("a and b must have equal shapes")
    return a * b + c

@mymuladd.register_fake
def _(a, b, c):
    torch._check(a.shape == b.shape)
    torch._check(a.dtype == b.dtype)
    torch._check(a.device == b.device)
    return torch.empty_like(a)
```

The equivalent schema definition in C++ is:

```
TORCH_LIBRARY(mylib, m) {
  m.def("mymuladd(Tensor a, Tensor b, float c) -> Tensor");
}
```

Step by step:

1. Choose a unique namespace and stable operator name.
2. Define the contract, especially mutation and aliasing.
3. Register one or more real implementations.
4. Register FakeTensor behavior so shape/dtype/device propagation works without real data.
5. Add an autograd formula if training needs gradients.
6. Add optional subsystem rules such as autocast or `vmap` only when required.
7. Use `torch.library.opcheck` for registration contracts and `torch.autograd.gradcheck` for derivative mathematics.

If an operation can be expressed entirely from built-in PyTorch operations and does not need an opaque boundary, an ordinary Python function is often better: the compiler and autograd already understand its pieces. A custom op is justified by non-PyTorch code, a custom kernel, or a required opaque semantic unit.

3. Important Subtopics

3.1 Namespace and operator identity

An operator is identified by `namespace::name`, optionally with overloads. A project-specific namespace prevents collisions.

- **Why it matters:** FX, export, dispatcher tables, and serialized representations need a stable identity.
- **Example:** `torch.ops.mylib.mymuladd.default` refers to the default overload.
- **Interview angle:** Do not register new project operations in reserved or generic namespaces such as `aten`.

3.2 Operator schema

The schema is a typed signature such as:

```
mymuladd(Tensor a, Tensor b, float c=0.0) -> Tensor
```

It can express optional values, lists, multiple results, keyword-only arguments, mutations, and aliases.

- **Why it matters:** PyTorch cannot safely transform an operation if its public behavior disagrees with its schema.
- **Example:** `Tensor(a!) x -> Tensor(a!)` marks mutation/aliasing for a conventional in-place shape.
- **Interview angle:** Schema is semantic metadata, not documentation only.

3.3 Functional, mutable, in-place, and `out=` operators

A functional operator does not mutate inputs and returns fresh tensors. Mutable operators change named inputs. Conventional in-place operators mutate and return the first tensor; `out=` variants write to keyword-only output tensors.

Form	Example intent	Key contract
Functional	<code>z = add(x, y)</code>	Inputs unchanged; fresh output
In-place	<code>add_(x, y)</code>	First tensor mutated and returned
<code>out=</code>	<code>add(x, y, out=z)</code>	Output buffer mutated/returned
Other mutation	Update state tensor	Mutation declared; arbitrary returned aliases avoided

- **Why it matters:** Functionalization, autograd, compilation, and FakeTensor rely on exact mutation and alias information.
- **Interview angle:** Prefer a functional design unless mutation is part of the required API.

3.4 Device kernels

The same schema can have different CPU and CUDA implementations.

```
TORCH_LIBRARY_IMPL(mylib, CPU, m) {
  m.impl("mymuladd", &mymuladd_cpu);
}

TORCH_LIBRARY_IMPL(mylib, CUDA, m) {
  m.impl("mymuladd", &mymuladd_cuda);
}
```

- **Why it matters:** Call sites stay device-independent while implementation differs.
- **Example:** CPU input chooses CPU; CUDA input chooses CUDA.
- **Interview angle:** A schema should normally be defined once; backend blocks add implementations rather than redefining it.

3.5 FakeTensor/Meta implementation

A fake implementation computes output metadata—shape, dtype, device, strides—without accessing real data or allocating real device storage.

- **Why it matters:** Compilers and exporters analyze programs symbolically.
- **Example:** `empty_like(a)` in a fake kernel produces correct metadata without running the CUDA kernel.
- **Interview angle:** A fake kernel must not dereference `data_ptr()` or branch on actual tensor values. Data-dependent output sizes need symbolic dynamic-size support or a graph break/constraint.

3.6 Decomposition versus opaque custom op

A decomposition describes an operation in terms of simpler PyTorch operators. An opaque custom op hides its internals behind one node.

- **Why it matters:** Decompositions let compilers optimize across the expression; opaque ops preserve external semantics but restrict visibility.
- **Example:** `a * b + c` as Python is compiler-visible, while `mylib::mymuladd` can remain one graph node.
- **Interview angle:** Use opacity only when it serves external-code or kernel integration; otherwise built-in composition is often easier.

3.7 Registration lifetime

Some Python registration APIs associate registrations with a `torch.library.Library` object. If a scoped registration object's lifetime ends, the registration can be removed.

- **Why it matters:** Tests or plugins may intentionally use scoped registrations; accidental garbage collection can make an op disappear.
- **Example:** Keep the library object alive at module scope when its registration should last for the process.
- **Interview angle:** Static C++ registrations normally run when the shared library loads; loading the library is therefore part of making the op available.

3.8 Testing with `opcheck`

`torch.library.opcheck` checks whether registrations obey framework contracts, including schema behavior, FakeTensor support, autograd registration, and compilation-related expectations.

```
sample = (
    torch.randn(8, device="cuda", requires_grad=True),
    torch.randn(8, device="cuda", requires_grad=True),
    0.5,
)
torch.library.opcheck(torch.ops.mylib.mymuladd.default, sample)
```

- **Why it matters:** Correct numerical output alone does not prove correct subsystem integration.
- **Interview angle:** `opcheck` does **not** prove the gradient formula is mathematically correct; use `gradcheck` separately.

3.9 Autocast and transforms

Mixed-precision autocast, `vmap`, tensor subclasses, and other transforms may require specific registrations.

- **Why it matters:** A forward kernel that works in eager float32 can fail or silently lose performance in a modern training pipeline.
- **Example:** An autocast rule can cast floating inputs to a selected dtype, run the op with autocast disabled, and return the result.
- **Interview angle:** Support only the transformations the product needs, but fail clearly when unsupported.

4. Real-World Example

A recommender system uses a proprietary GPU library for compressed embedding lookup. An ordinary Python wrapper causes a graph break during `torch.compile`. Registering the lookup as `recommend::compressed_lookup` gives it:

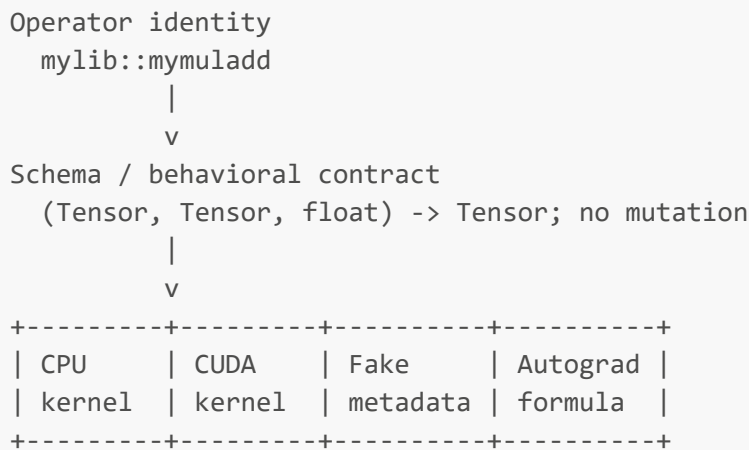
- a CUDA implementation that calls the library;
- a CPU reference for testing;
- a fake implementation that predicts `[batch, embedding_dim]`;
- an autograd formula if embeddings are trainable;
- an autocast policy if the kernel supports reduced precision.

```
Model graph
|
+--> aten::linear
|
+--> recommend::compressed_lookup <-- opaque but understood node
|
+--> aten::relu
```

The compiler does not need to understand the proprietary code. It needs an accurate contract and metadata rule so it can place the node in a valid graph.

5. Diagrams / Mental Models

Registration stack



Contract-first decision

```

Can built-in PyTorch operations express it?
|
+-- yes --> Need opaque identity or external code boundary?
|           +-- no --> ordinary Python function/decomposition
|           +-- yes -> custom operator
|
+-- no  --> custom kernel/library -> custom operator

```

6. Common Interview Questions

1. What is a custom operator in PyTorch?

- **Answer:** A named, schema-defined operation registered with one or more implementations and optional subsystem rules.
- **Expected:** Stable identity, contract, dispatch.
- **Common mistake:** Calling any Python function a custom operator.

2. When should you create one?

- **Answer:** To wrap external code, expose a handwritten kernel, add missing semantics, or create an intentional opaque graph boundary.
- **Expected:** Prefer normal composition when built-in ops are sufficient.
- **Common mistake:** Registering every convenience function as an op.

3. What does the schema describe?

- **Answer:** Argument/result types, defaults, optionality, lists, mutation, and aliasing.
- **Expected:** It is enforced semantic information used by transformations.
- **Common mistake:** Treating it like a type hint with no runtime consequences.

4. Why use a project namespace?

- **Answer:** To prevent name collisions and provide a stable owner for operator identity.
- **Expected:** Example `mylib::op`.
- **Common mistake:** Defining custom operators in `aten`.

5. What is a fake implementation?

- **Answer:** A metadata-only rule that returns fake output tensors with correct shape/dtype/device without reading real values.
- **Expected:** Used by compilation, export, and symbolic tracing.
- **Common mistake:** Calling the actual CUDA kernel or inspecting data in it.

6. What is the difference between functional and in-place custom ops?

- **Answer:** Functional ops leave inputs unchanged and return fresh storage; in-place ops mutate and conventionally return the mutated first tensor.
- **Expected:** Mutation/aliasing must be declared accurately.
- **Common mistake:** Marking an op functional while mutating an input.

7. How are CPU and CUDA implementations attached to one op?

- **Answer:** Define one schema and register kernels under CPU and CUDA dispatch keys.
- **Expected:** Same semantic contract across backends.
- **Common mistake:** Defining separate public operator names solely for CPU and CUDA.

8. What does `torch.library.opcheck` verify?

- **Answer:** Registration and subsystem contracts such as schema agreement, fake behavior, autograd registration, and compilation compatibility for sample inputs.
- **Expected:** Run representative inputs and devices.
- **Common mistake:** Claiming it proves derivative mathematics.

9. What is the difference between `opcheck` and `gradcheck`?

- **Answer:** `opcheck` validates operator registration behavior; `gradcheck` compares analytical gradients with numerical finite differences.
- **Expected:** Use both for a trainable custom op.
- **Common mistake:** Substituting one for the other.

10. Why does `torch.compile` care about custom-op metadata?

- **Answer:** It needs output shapes/dtypes, mutation/aliasing, and graph semantics without executing real data kernels during tracing.
- **Expected:** Mention FakeTensor and functionalization.
- **Common mistake:** Assuming successful eager execution guarantees compiled execution.

7. Deep-Dive Questions

1. Why can a false “no mutation” declaration cause silent incorrectness?

- Compilers may reorder, eliminate, or functionalize calls based on the declared contract. Hidden mutation violates those assumptions, so graph behavior can diverge from eager behavior.

2. How should a fake kernel handle a data-dependent output shape such as `nonzero`?

- It cannot inspect values. It must create a symbolic dynamic size through the fake implementation context when supported, or the operator must impose a constraint/unsupported boundary.

3. Why can arbitrary returned aliases be difficult for custom-op systems?

- Transformations need exact storage relationships. Conventional functional, in-place, and `out=` patterns are tractable; arbitrary views and mutation combinations require richer alias semantics and can break functionalization.

4. Should CPU and CUDA kernels be allowed to have different semantics?

- They may differ within documented numerical tolerance, but shape, dtype policy, mutation, errors, and mathematical meaning should match. Otherwise device movement changes the program's meaning.

5. What is the stable identity benefit during export?

- The exported graph can carry `namespace::op` as an explicit dependency. The deployment runtime must provide a compatible implementation and schema for that identity.

8. Comparison Tables

Ordinary Python function	Custom operator
Compiler can inspect composed PyTorch ops	Usually an opaque named node
Autograd comes from constituent ops	Requires an autograd formula when implementation is opaque
No schema/dispatch registration	Explicit schema and registrations
Best for ordinary model logic	Best for external/native kernels and semantic boundaries
Easy to change	Stable identity becomes an API contract
Schema	Kernel
Defines what the op means at its boundary	Implements the computation for a context
Device-independent contract	Often device/key-specific
Declared once	May have CPU, CUDA, Fake, Autograd variants
Includes mutation/aliasing	Must obey the schema
Real kernel	Fake/Meta implementation
Reads actual tensor values	Must not read actual values

Real kernel	Fake/Meta implementation
Performs computation	Computes metadata only
Allocates real storage/output	Produces fake/meta tensor descriptions
Used at execution	Used during analysis/tracing/compilation

9. Common Mistakes

- Creating a custom op when a normal composition of built-in ops is enough.
- Using a global or reserved namespace.
- Declaring no mutation while changing an input or global tensor state.
- Returning an input alias from a supposedly functional op.
- Registering a CUDA kernel but no FakeTensor implementation for compiler use.
- Reading tensor data inside a fake implementation.
- Defining the same schema independently in multiple translation units.
- Assuming successful eager calls imply `torch.compile`, export, autocast, or `vmap` support.
- Using `opcheck` as a numerical correctness or gradient proof.
- Loading an extension too late, after code tries to resolve `torch.ops.mylib.*`.

10. Edge Cases / Special Cases

- **Optional tensors:** A schema must distinguish missing values from empty tensors.
- **Multiple outputs:** Fake and backward rules must return matching structures and counts.
- **Non-tensor arguments:** Device dispatch usually comes from tensor arguments; tensor-free factory-like ops need deliberate backend selection.
- **No tensor inputs:** The dispatcher cannot infer a backend from an absent tensor key set in the usual way.
- **Mutation plus views:** Alias relationships are difficult; use conventional supported patterns and test functionalization.
- **Data-dependent shapes:** Fake execution cannot inspect data values.
- **Symbolic shapes:** Avoid converting symbolic dimensions to ordinary Python integers inside metadata rules.
- **Autocast:** Reduced-precision inputs may reach a kernel that only supports float32 unless a policy is registered.
- **Serialization/export:** Deployment must load a compatible registration for the exported operator identity.
- **Duplicate registration:** Two libraries registering the same schema/kernel key can cause load-time errors.
- **Registration lifetime:** Scoped Python registrations can disappear when their owner is destroyed.

11. How to Explain in Interview

“A PyTorch custom operator is a stable `namespace::name` plus a schema and registrations. The schema declares types, mutation, and aliasing; CPU and CUDA keys provide real kernels; a FakeTensor rule provides output metadata for compilation; and an autograd formula provides training support. I use a normal Python composition when built-in ops already express the work, and create a custom op for a native kernel, external library, or intentional opaque graph boundary.”

12. Quick Revision Notes

- Custom operator = identity + schema + registrations.
- Schema is a behavioral contract, especially for mutation and aliasing.
- One schema can route to CPU and CUDA kernels.
- Fake implementation computes metadata without data access.
- Functional ops are simplest for transformations.
- Opaque op restricts compiler visibility; decomposition exposes internals.
- `opcheck`: registration correctness.
- `gradcheck`: mathematical derivative correctness.
- Namespace avoids collisions and identifies ownership.
- Interview trap: a C++/Python function is not automatically a registered operator.

13. Practice Tasks

1. Define `mylib::mymuladd` with `torch.library.custom_op` and `mutates_args=()`.
2. Register a FakeTensor implementation and inspect it under `FakeTensorMode`.
3. Register separate CPU and CUDA implementations for the same schema.
4. Run `opcheck` with CPU/CUDA, `requires_grad` on/off, empty, scalar, and non-contiguous samples.
5. Deliberately mutate an input while declaring the op functional; observe which checks fail.
6. Create a multi-output op and make its fake result match the exact output structure.
7. Compare an ordinary function and an opaque custom op in an exported/FX graph.
8. Design schemas for functional, in-place, and `out=` versions of addition and explain their alias annotations.

14. Final Cheat Sheet

Item	Must remember
Core definition	Named, schema-defined operation with registered behavior
Identity	<code>namespace::operator[.overload]</code>
Contract	Types, defaults, mutation, aliasing, outputs
Real implementations	CPU, CUDA, or other backend kernels
Compiler support	Fake/Meta implementation with no data access
Training support	Registered autograd formula
Key tests	<code>opcheck</code> + reference tests + <code>gradcheck</code>
Most asked	Schema, fake kernels, mutation, custom op vs function
Main trap	Registration correctness and numerical correctness are separate
One-line answer	"A custom op gives PyTorch a stable semantic node with an explicit contract and backend-specific behavior."

PyTorch Dispatcher

1. Overview

The **PyTorch dispatcher** is the runtime routing system that decides which implementation of an operator should run for a particular call. When code invokes an operation such as addition or `mylib::mymuladd`, the dispatcher combines the operator identity with information carried by the arguments and active modes, then selects an appropriate registered kernel.

Device type is only one routing dimension. The dispatcher also coordinates autograd, autocast, functionalization, batching transforms, tensor subclasses, and backend extensions. A CUDA tensor therefore does not simply trigger an `if tensor.is_cuda()` branch in every operator.

The dispatcher matters because it gives PyTorch one public operation with many composable behaviors. It is used on virtually every operator call in eager PyTorch and underpins custom backend and tensor-transform integration. Interviewers ask about it to test whether candidates understand dynamic dispatch, separation of interface from implementation, key precedence, fallbacks, and how device/autograd behavior composes.

2. Core Idea

Think of the dispatcher as an airport control tower. The flight number is the operator name; the passengers and current conditions contribute dispatch keys; and the registry says which runway procedure applies. The tower selects the highest-priority applicable rule, which may do work and redispach to the next rule.

```

Call: torch.ops.mylib.mymuladd(a_cuda_requires_grad, b_cuda, 0.5)
      |
      v
Operator handle + argument key set
{AutogradCUDA, CUDA, ...}
      |
      v
Highest-priority matching registration
autograd handling / formula
      |
      redispatch
      v
CUDA kernel
  
```

At registration time:

```

TORCH_LIBRARY(mylib, m) {
  m.def("mymuladd(Tensor a, Tensor b, float c) -> Tensor");
}

TORCH_LIBRARY_IMPL(mylib, CPU, m) {
  m.impl("mymuladd", &mymuladd_cpu);
}
  
```

```
TORCH_LIBRARY_IMPL(mylib, CUDA, m) {  
  m.impl("mymuladd", &mymuladd_cuda);  
}
```

At call time:

1. Python resolves `mylib::mymuladd` to an operator handle.
2. The dispatcher builds a dispatch-key set from tensor arguments and active modes.
3. It chooses the highest-priority key with a registered kernel or fallback.
4. A wrapper-like key may add behavior and redispach after excluding itself.
5. Eventually a backend kernel such as CPU or CUDA performs the numerical work.
6. The result travels back through any wrappers to the caller.

3. Important Subtopics

3.1 Operator handle and schema lookup

The dispatcher first needs the exact operator and overload. The schema validates the call and defines the available entry.

- **Why it matters:** Dispatch does not choose between unrelated operator names; it chooses among implementations of one schema.
- **Example:** `aten::add.Tensor` and `aten::add.Scalar` are overloads with different signatures.
- **Interview angle:** Overload resolution and dispatch-key selection are different stages.

3.2 Dispatch keys

A dispatch key represents a routing concern or implementation family, such as CPU, CUDA, Autograd, Autocast, Functionalize, or a custom backend.

- **Why it matters:** Keys avoid hard-coded device/type switches scattered through operators.
- **Example:** A CUDA tensor contributes a CUDA-related backend key; grad mode and tensor properties can activate autograd behavior.
- **Interview angle:** Actual key names and ordering are implementation details that can evolve; the durable concept is an ordered key set with registrations and fallbacks.

3.3 Dispatch-key set and precedence

A call may carry multiple applicable keys. The dispatcher uses a defined priority order, not “first registered wins.”

- **Why it matters:** Transforming layers such as autocast or autograd may need to wrap the backend kernel.
- **Example:** Autocast can convert input dtype, disable its own key, and redispach to CUDA.
- **Interview angle:** A tensor can be both CUDA-backed and autograd-participating; these are not mutually exclusive choices.

3.4 Backend keys

Backend keys select the storage/execution implementation, such as CPU or CUDA.

- **Why it matters:** One operator schema remains portable across devices.
- **Example:** `mymuladd` with CPU tensors selects `mymuladd_cpu`; CUDA tensors select `mymuladd_cuda`.
- **Interview angle:** Validate that tensor arguments do not imply conflicting backend keys, such as CPU `a` and CUDA `b`, unless cross-device semantics are explicitly supported.

3.5 Wrapper/functionality keys

Some keys add behavior around another implementation rather than replacing the backend computation.

- **Why it matters:** Autograd, autocast, functionalization, and batching can compose with CPU/CUDA kernels.
- **Example:** An autocast rule casts float32 inputs to float16, then redispaches with autocast disabled to prevent recursion.
- **Interview angle:** Explain “intercept, transform/record, redispach” rather than imagining one final device switch.

3.6 Redispatch

Redispatch means calling the same operator again with the current key removed/excluded, allowing the next applicable registration to run.

- **Why it matters:** Wrapper keys would recursively call themselves forever without exclusion.
- **Example:** An autograd wrapper records backward information, then redispaches to the device kernel for the forward value.
- **Interview angle:** Redispatch preserves the remaining key set; directly calling a backend function bypasses other dispatcher layers.

3.7 Catch-all and composite implementations

A device-agnostic implementation written from other PyTorch operators can sometimes be registered without separate CPU/CUDA kernels. Composite registrations allow reuse of existing operators and their behavior.

- **Why it matters:** One implementation may work across devices and inherit autograd through its constituent operations.
- **Example:** `return a * b + c` can run wherever multiplication and addition run.
- **Interview angle:** Composite implementation is convenient but does not create a fused kernel; dispatching constituent operations still occurs.

3.8 Backend fallback

A backend fallback handles many operators for a key when no per-operator kernel exists, often to wrap, redispach, or report a consistent error.

- **Why it matters:** Cross-cutting backends/modes need not register identical glue for every operator.
- **Example:** A fallback for a tensor wrapper may unwrap arguments, redispach, then wrap outputs.
- **Interview angle:** A fallback is broader and less specific than a per-operator registration.

3.9 Boxed and unboxed calling

An unboxed call uses a normal typed C++ function signature. A boxed call passes values in a generic stack-like representation.

- **Why it matters:** Unboxed calls are type-safe and efficient; boxed calls make generic fallbacks possible across many schemas.
- **Example:** A logging fallback can inspect a generic argument stack without compile-time knowledge of every operator signature.
- **Interview angle:** Boxing is an implementation technique, not Python boxing and not device transfer.

3.10 Inspecting dispatch

PyTorch provides diagnostic APIs and environment-level tooling to inspect operator schemas and dispatch tables, although exact internal helpers are not stable public APIs.

- **Why it matters:** “No kernel found” errors become easier to diagnose when you can see which key was selected and which registrations exist.
- **Example:** Check that the shared library loaded before expecting a CUDA registration.
- **Interview angle:** Diagnose missing schema, missing backend kernel, wrong active key, and registration lifetime as separate problems.

4. Real-World Example

Consider one model executed in four situations:

1. CPU inference in float32.
2. CUDA inference under autocast.
3. CUDA training with gradients.
4. Symbolic tracing with FakeTensors.

The source code can call the same operator each time:

```
y = torch.ops.mylib.mymuladd(a, b, 0.5)
```

Conceptually, routing differs:

CPU inference:	CPU -> result
CUDA autocast:	Autocast -> cast/redispatch -> CUDA -> result
CUDA training:	Autograd -> record/redispatch -> CUDA -> result + grad edge
Fake tracing:	Fake/Meta rule -> metadata-only result

This is why dispatch registration is more scalable than a long `if/elif` chain inside the Python model.

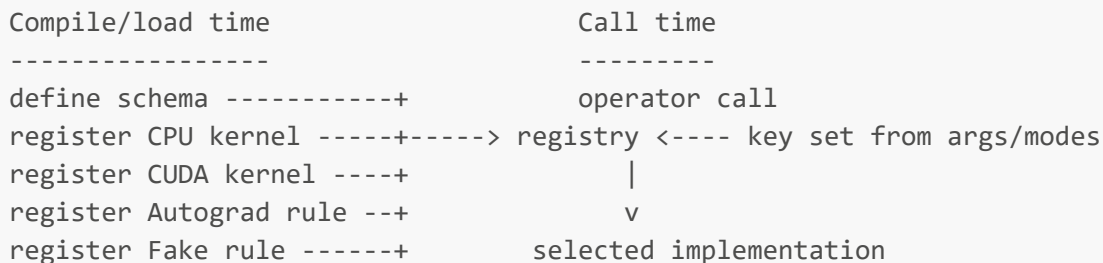
5. Diagrams / Mental Models

Two-dimensional intuition

Execution backend			
	CPU	CUDA	Other
Eager	CPU impl	CUDA impl	...
Autograd	wrapper -> backend implementation		
Autocast	cast ----> backend implementation		
Fake	metadata-only implementation		

The real dispatcher uses an ordered key system rather than a literal matrix, but this diagram helps separate **where data lives** from **which behavior wraps execution**.

Registration and invocation



6. Common Interview Questions

1. What does the PyTorch dispatcher do?

- **Answer:** It selects the registered implementation of an operator based on the operator identity, arguments, and active dispatch modes/keys.
- **Expected:** More than device routing.
- **Common mistake:** Describing a single `is_cuda()` condition.

2. What is a dispatch key?

- **Answer:** A routing label representing a backend or cross-cutting behavior such as CUDA, Autograd, or Autocast.
- **Expected:** Multiple keys can apply to one call.
- **Common mistake:** Treating keys as tensor dtypes.

3. How does the dispatcher choose between CPU and CUDA kernels?

- **Answer:** Tensor arguments contribute backend keys; the highest-priority applicable registered key ultimately routes to the matching backend implementation.
- **Expected:** Inputs should agree on supported device semantics.
- **Common mistake:** Saying Python manually selects the kernel.

4. Can more than one dispatch behavior participate in one call?

- **Answer:** Yes. Wrapper-like keys can intercept and redispatch to other keys, such as autocast leading to CUDA.
- **Expected:** Ordered composition.
- **Common mistake:** Assuming exactly one key exists.

5. What is redispatch?

- **Answer:** Reinvoking the same operator after removing/excluding the current key so the next applicable implementation runs.
- **Expected:** Prevent recursion and preserve other layers.
- **Common mistake:** Directly calling a CUDA function and claiming it is equivalent in all contexts.

6. What happens if no kernel is registered for the selected key?

- **Answer:** A suitable fallback/composite path may handle it; otherwise PyTorch raises a missing-kernel error with available registrations.
- **Expected:** Loading/registration can be the issue, not only implementation absence.
- **Common mistake:** Silently falling back to CPU and copying tensors.

7. What is a backend fallback?

- **Answer:** A key-wide handler used when an operator lacks a more specific registration for that key.
- **Expected:** Useful for wrappers/custom backends; broader than per-op kernels.
- **Common mistake:** Calling it the same as a CPU fallback.

8. What is a composite implementation?

- **Answer:** An implementation expressed using other PyTorch operators so it can reuse their backend/autograd support.
- **Expected:** Portable but not necessarily fused.
- **Common mistake:** Assuming one composite function equals one GPU kernel.

9. What is boxed dispatch?

- **Answer:** Generic invocation through a runtime value stack, useful for schema-independent fallbacks and tooling.
- **Expected:** Contrast with typed/unboxed calls.
- **Common mistake:** Confusing it with Python object boxing or tensor allocation.

10. Why is dispatch useful for custom operators?

- **Answer:** It preserves one public schema while allowing CPU/CUDA kernels and autograd, fake, autocast, or transform-specific behavior.
- **Expected:** Extensibility without conditionals at call sites.
- **Common mistake:** Registering separate user-facing names for every device.

7. Deep-Dive Questions

1. How can an autograd key and a CUDA key both be relevant?

- They describe different concerns. Autograd determines gradient recording/behavior; CUDA determines the numerical backend. An autograd layer can redispach to CUDA for the forward computation.

2. Why is dispatch priority important?

- If a backend kernel ran before autocast or functionalization, cross-cutting semantics could be skipped. Priority establishes a consistent wrapper order.

3. Why not put all behavior inside every CPU and CUDA kernel?

- It duplicates autograd, autocast, batching, and wrapper logic across backends. Dispatch keys separate orthogonal concerns and allow them to compose.

4. How can tensor subclasses intercept operators?

- Tensor subclass protocols participate near the dispatcher boundary, allowing custom behavior around operator calls. Correct custom-op schemas and registrations make these interactions more predictable.

5. What problem occurs when a wrapper redispaches without excluding itself?

- The same wrapper remains the highest-priority key and calls itself recursively, causing infinite recursion or stack overflow.

8. Comparison Tables

Compile-time C++ overload resolution	PyTorch runtime dispatch
Chosen by static argument types	Chosen from runtime operator/key information
Compiler selects function	Dispatcher registry selects kernel
Closed over compiled overload set	Extensible through registrations
Does not understand tensor device modes	Coordinates backend and framework behavior
Backend key	Wrapper/functionality key
Selects execution/storage family	Adds cross-cutting behavior
CPU, CUDA, custom device	Autograd, autocast, functionalization-like behavior
Usually performs numerical work	Often transforms/records then redispaches
Endpoint of many dispatch paths	Layer around another implementation
Per-operator kernel	Backend fallback
Specific to one schema/key	Applies broadly for a key
Knows typed operation semantics	Often generic/boxed
Preferred specialized behavior	Used when no more specific kernel exists

9. Common Mistakes

- Reducing the dispatcher to CPU-versus-CUDA selection.
- Confusing operator overload resolution with dispatch-key selection.
- Assuming registration order defines dispatch priority.
- Calling a backend function directly from a wrapper and bypassing required layers.
- Redispatching without excluding the active wrapper key.
- Defining schemas more than once instead of adding implementations.
- Assuming unsupported CUDA input silently moves to CPU.
- Ignoring autograd, FakeTensor, or functionalization behavior.
- Treating composite registration as kernel fusion.
- Debugging “operator does not exist” as if it were the same as “kernel missing for CUDA.”

10. Edge Cases / Special Cases

- **Mixed-device arguments:** The combined key set may be invalid for the operator; explicit same-device checks give clearer errors.
- **No tensor arguments:** There may be no backend key to infer, so factory-like operations need explicit device-aware design.
- **Optional tensor is None:** It contributes no tensor key.
- **Empty tensors:** They still carry device/dtype keys even though they contain no elements.
- **Tensor subclasses/modes:** They can intercept calls and change which layer is reached first.
- **Autograd disabled:** `no_grad` changes recording needs, but tensor backend routing still occurs.
- **Inference mode:** Has stronger semantics than merely disabling gradient recording.
- **Autocast nested/disabled:** Wrapper rules must avoid self-recursion and honor current state.
- **Missing shared-library load:** The schema may not exist at all, even though extension code compiled successfully.
- **Duplicate registrations:** A second kernel for the same operator/key can cause registration errors unless an API explicitly permits controlled override.
- **Internal keys:** Exact names and priorities can change; avoid depending on undocumented internals in interview designs and production APIs.

11. How to Explain in Interview

“The PyTorch dispatcher maps one operator schema to the right registered behavior at runtime. Tensor arguments contribute backend keys like CPU or CUDA, while modes such as autograd or autocast add wrapper-like behavior. The dispatcher follows key priority; a wrapper may record or transform inputs and redispatch to the next key until a backend kernel performs the computation. This avoids hard-coded device checks and lets orthogonal features compose.”

12. Quick Revision Notes

- Input to dispatch: operator identity + runtime key set.
- Output: selected registered kernel/fallback.
- CPU/CUDA are backend concerns.
- Autograd/autocast/functionalization are cross-cutting concerns.
- Multiple keys may apply; priority chooses the next action.
- Wrapper keys often redispatch after excluding themselves.
- Composite implementation reuses other ops; it is not fusion.
- Boxed calls enable generic handlers; unboxed calls are typed.

- One schema, many implementations.
- Interview trap: dispatch is broader than device selection.

13. Practice Tasks

1. Register `mymuladd` for CPU only and observe the error for CUDA input.
2. Add the CUDA registration without redefining the schema.
3. Call the operator with CPU `a` and CUDA `b`; design a clear contract error.
4. Write a Python autocast registration and confirm the dtype reaching the base kernel.
5. Compare a composite implementation with separate CPU/CUDA kernels.
6. Inspect an operator's schema and dispatch table using the diagnostic facilities available in your installed PyTorch version.
7. Draw the conceptual key path for CPU inference, CUDA training, CUDA autocast, and FakeTensor tracing.
8. Explain how a generic logging fallback could use boxed arguments.

14. Final Cheat Sheet

Item	Must remember
Core definition	Runtime router from operator call to registered behavior
Dispatcher input	Operator/overload + ordered key set
Backend examples	CPU, CUDA, other devices
Wrapper examples	Autograd, autocast, functionalization, transforms
Composition tool	Redispatch after excluding the current key
Broad handler	Backend fallback
Generic call form	Boxed stack; typed fast form is unboxed
Most asked	Keys, precedence, redispatch, fallback, composite kernels
Main trap	One call can involve several dispatch layers, not one device switch
One-line answer	"The dispatcher composes runtime concerns and routes a named op to the correct registered implementation."

Autograd Integration

1. Overview

Autograd integration makes a custom operator participate correctly in PyTorch's reverse-mode automatic differentiation. The forward kernel computes outputs. The autograd formula explains how an incoming gradient with respect to each output becomes gradients with respect to differentiable inputs.

For the running operation

$$y = a * b + c$$

the local derivatives are:

$$dy/da = b \quad dy/db = a \quad dy/dc = 1$$

Because `c` is a Python scalar in the schema, autograd returns tensor gradients only for `a` and `b`; no tensor gradient is returned for `c`.

Autograd integration matters whenever a custom kernel appears in model training, gradient-based optimization, or higher-order differentiation. Real systems use it for fused layers, differentiable simulators, rendering, scientific operators, and external libraries. Interviewers ask about it to test the chain rule, computation graphs, saved state, in-place safety, gradient validation, and the distinction between a correct forward kernel and a correct training operator.

2. Core Idea

Think of the forward pass as walking through a city while leaving only the breadcrumbs needed to return. The backward pass starts with a gradient from later operations and follows local derivative rules in reverse. Saving every intermediate wastes memory; saving too little makes the return path impossible.

For a registered custom operator, the recommended registration shape is:

```
import torch

def setup_context(ctx, inputs, output):
    a, b, c = inputs
    ctx.save_for_backward(a, b)

def backward(ctx, grad_out):
    a, b = ctx.saved_tensors
    grad_a = grad_out * b if ctx.needs_input_grad[0] else None
    grad_b = grad_out * a if ctx.needs_input_grad[1] else None
    return grad_a, grad_b, None

torch.library.register_autograd(
    "mylib::mymuladd",
    backward,
    setup_context=setup_context,
)
```

Step by step:

1. The forward call produces `y` using the CPU or CUDA implementation.

2. If gradient recording is needed, `setup_context` saves `a` and `b` because the derivative formula uses them.
3. Later, a scalar loss sends `grad_out = dL/dy` into this node.
4. The backward formula applies the chain rule:
 - $dL/da = grad_out * b$
 - $dL/db = grad_out * a$
5. It returns one entry per forward input: tensor gradients for `a` and `b`, and `None` for scalar `c`.
6. Autograd accumulates those gradients into earlier graph paths and ultimately into leaf `.grad` fields when `.backward()` is used.

The backward and context setup should use PyTorch operations that autograd/compiler transforms can trace. If the backward itself requires opaque native code, expose that work as another registered custom operator and call it from the backward formula.

3. Important Subtopics

3.1 Reverse-mode automatic differentiation

Reverse mode computes vector-Jacobian products (VJPs): given `grad_out`, it returns `grad_out @ J` without materializing the full Jacobian.

- **Why it matters:** Neural networks often have millions of parameters but a scalar loss, where reverse mode is efficient.
- **Example:** Elementwise multiply uses `grad_out * b` rather than creating a diagonal Jacobian.
- **Interview angle:** Backward receives the upstream gradient; local derivatives alone are incomplete without the chain rule.

3.2 Computation graph and gradient function

During a recorded forward pass, PyTorch connects outputs to nodes describing how to propagate gradients to inputs.

- **Why it matters:** Autograd traverses graph edges in reverse topological order.
- **Example:** `loss = mymuladd(a, b, c).sum()` adds a sum node after the custom-op node.
- **Interview angle:** Gradients accumulate when a tensor influences the loss through multiple paths.

3.3 `setup_context` and saved tensors

`setup_context` stores tensors or metadata required by backward. Tensors should be saved with `ctx.save_for_backward`.

- **Why it matters:** Saved-tensor machinery supports lifecycle checks and memory-management features.
- **Example:** Save `a` and `b`; there is no reason to save output `y` for the multiply-add derivative.
- **Interview angle:** Save the minimum required state, but do not recompute expensive or nondeterministic values unless that trade-off is deliberate.

3.4 Backward signature and return contract

Backward receives one gradient per output and returns one gradient slot per forward input. Non-tensor or non-differentiable inputs get `None`.

- **Why it matters:** Wrong arity or order attaches gradients to the wrong inputs.
- **Example:** (`grad_a`, `grad_b`, `None`) matches (`a`, `b`, `c`).
- **Interview angle:** Multiple outputs produce multiple `grad_out` arguments; some may be `None` if unused depending on materialization settings.

3.5 Broadcasting and reduction in backward

If the forward allows broadcasting, a gradient for a broadcasted input must be reduced back to that input's original shape.

- **Why it matters:** `grad_out * b` can have output shape, not necessarily `a.shape`.
- **Example:** If `a` is `[N, D]` and bias `b` is `[D]`, the bias gradient sums across the `N` dimension.
- **Interview angle:** Use a sum-to-size operation or equivalent; never return a broadcasted gradient with the wrong shape.

3.6 `requires_grad`, leaf tensors, and accumulation

`requires_grad=True` requests graph tracking for differentiable operations. Leaf tensors normally receive accumulated `.grad` values after `.backward()`.

- **Why it matters:** A non-leaf result can require grad but not retain `.grad` by default.
- **Example:** Calling backward twice without clearing leaf gradients accumulates into them.
- **Interview angle:** `requires_grad` does not mean every tensor automatically stores a `.grad` field.

3.7 In-place operations and version counters

PyTorch tracks mutations using version counters. If a value saved for backward changes in place, autograd may reject backward because the derivative would use corrupted state.

- **Why it matters:** Silent mutation could produce silently wrong gradients.
- **Example:** Modifying saved `a` after forward can invalidate `grad_b = grad_out * a`.
- **Interview angle:** Do not bypass mutation tracking with raw pointers or `.data`; declare mutation accurately and design functional ops where possible.

3.8 Higher-order gradients

A backward formula written with differentiable PyTorch operations can itself form a graph when `create_graph=True`, enabling second derivatives.

- **Why it matters:** Meta-learning, gradient penalties, and some scientific workloads use higher-order derivatives.
- **Example:** A backward implemented by an opaque, non-differentiable raw CUDA call will not automatically support double backward.
- **Interview angle:** First-order correctness does not imply second-order support; test it if promised.

3.9 `register_autograd` versus `torch.autograd.Function`

`torch.autograd.Function` defines a custom forward/backward pair for Python-level autograd use. For an operator already registered with `torch.library` or C++ operator APIs,

`torch.library.register_autograd` attaches the derivative rule directly to that operator identity and is the preferred integration path.

- **Why it matters:** Operator registration remains visible and composes more reliably with PyTorch transforms and compilation.
- **Example:** Register the formula for `mylib::mymuladd` rather than hiding the op behind an unrelated `Function.apply` wrapper.
- **Interview angle:** `autograd.Function` remains useful for custom differentiation logic that is not an already-registered custom operator; do not stack mechanisms without understanding transform behavior.

3.10 Gradient testing

`torch.autograd.gradcheck` perturbs double-precision inputs and compares numerical finite-difference gradients with the analytical backward formula.

```
def f(a, b):
    return torch.ops.mylib.mymuladd(a, b, 0.25)

a = torch.randn(4, dtype=torch.double, requires_grad=True)
b = torch.randn(4, dtype=torch.double, requires_grad=True)
assert torch.autograd.gradcheck(f, (a, b))
```

- **Why it matters:** A backward can have the correct shape and still be mathematically wrong.
- **Example:** Accidentally returning `grad_out * a` for `grad_a` may pass simple shape tests but fail `gradcheck`.
- **Interview angle:** Use double precision, smooth test points, appropriate tolerances, and separate `opcheck` from `gradcheck`.

3.11 Non-differentiable and integer outputs

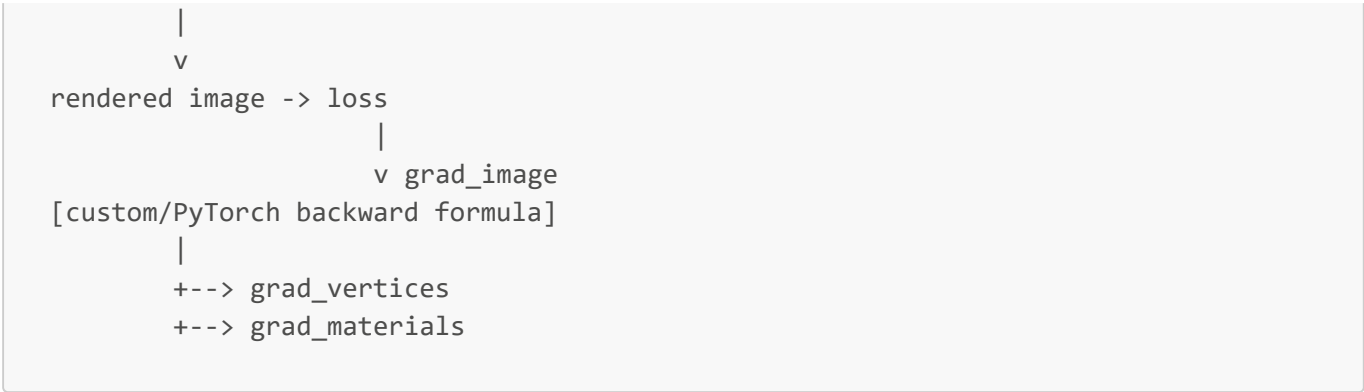
Some outputs are indices, masks, or discrete decisions and should not receive gradients.

- **Why it matters:** Autograd cannot meaningfully differentiate every operation.
- **Example:** Argmax indices are non-differentiable even if derived from floating input.
- **Interview angle:** Mark non-differentiable outputs or return `None` appropriately rather than inventing a gradient.

4. Real-World Example

A differentiable renderer uses a custom CUDA kernel to rasterize triangles. The forward pass outputs pixel colors and may save visibility/barycentric information. The backward pass uses the upstream image gradient to compute gradients for vertex positions and material parameters.

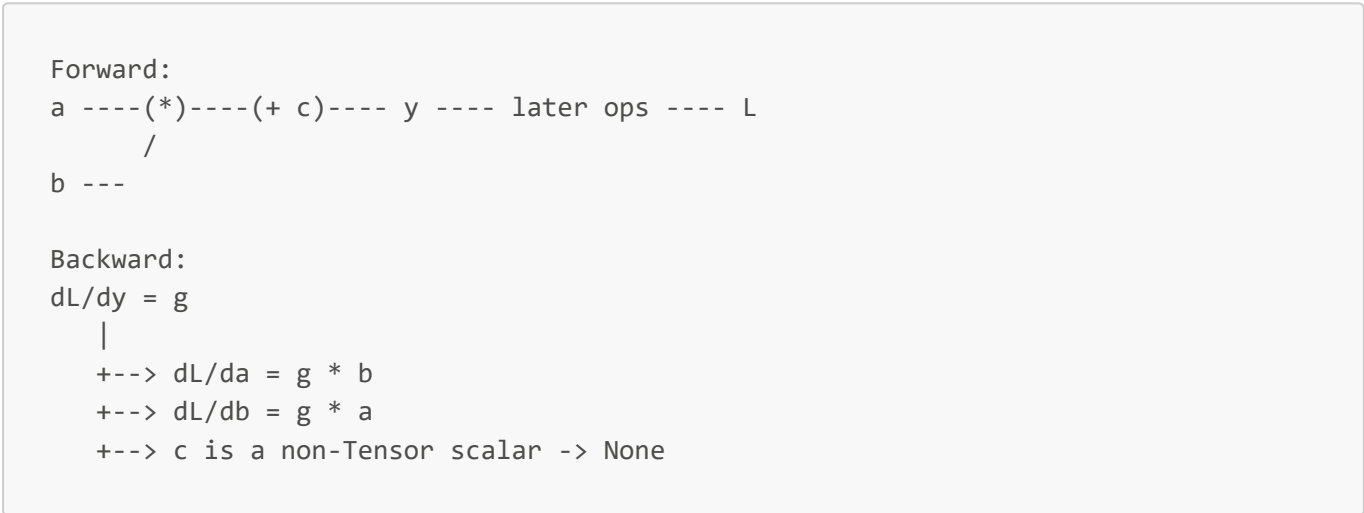
```
vertices, materials
  |
  v
[custom CUDA rasterizer] --save--> visibility/barycentric state
```



The engineering trade-off is saved state versus recomputation. Saving every per-pixel intermediate consumes GPU memory; recomputing visibility in backward costs compute and may be nondeterministic if tie-breaking differs. A good implementation documents and tests that choice.

5. Diagrams / Mental Models

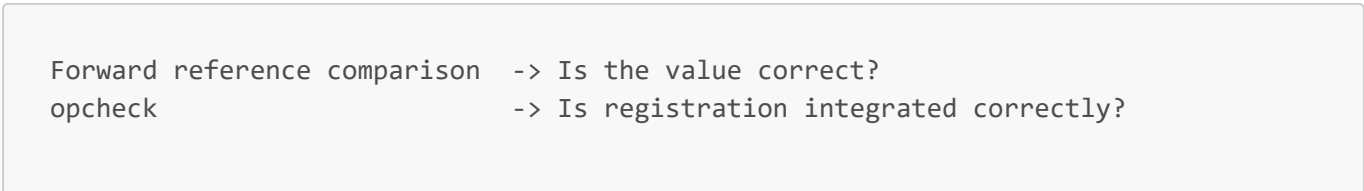
Chain rule through the running op



Save-versus-recompute trade-off

Choice	Forward memory	Backward compute	Risks
Save tensors/intermediates	Higher	Lower	Memory pressure, version checks
Recompute in backward	Lower	Higher	Cost, nondeterminism, must reproduce semantics
Save compact metadata	Medium/low	Medium	More complex formula

Gradient validation layers



gradcheck	-> Is the first derivative numerically correct?
gradgradcheck	-> Is the second derivative numerically correct?

6. Common Interview Questions

1. How does autograd work at a high level?

- **Answer:** It records differentiable operations during forward and traverses the graph backward, applying local VJPs via the chain rule.
- **Expected:** Upstream gradient, reverse topological traversal, accumulation.
- **Common mistake:** Saying it symbolically differentiates the whole Python program.

2. What must a custom operator provide for training?

- **Answer:** A correct autograd formula, plus any forward values saved in context; the base operator must also have valid registrations.
- **Expected:** Prefer `torch.library.register_autograd` for a registered custom op.
- **Common mistake:** Assuming a CUDA forward kernel automatically produces gradients.

3. What is the backward formula for $y = a * b + c$?

- **Answer:** Given $g = dL/dy$, return $g*b$ for a , $g*a$ for b , and `None` for scalar c .
- **Expected:** Include the upstream gradient.
- **Common mistake:** Returning only b and a .

4. Why save tensors for backward?

- **Answer:** The derivative formula may need forward values that will not otherwise be available during reverse traversal.
- **Expected:** Save only necessary state with `save_for_backward`.
- **Common mistake:** Storing every intermediate or keeping unsafe raw pointers.

5. Why does backward return `None` for some inputs?

- **Answer:** The input may be non-tensor, non-differentiable, or not require a gradient.
- **Expected:** Return slots align with forward input order.
- **Common mistake:** Returning a zero tensor of an arbitrary shape for a scalar argument.

6. How does broadcasting affect gradients?

- **Answer:** The gradient for a broadcasted input must sum over expanded dimensions to match the original input shape.
- **Expected:** Mention sum-to-size/unbroadcasting.
- **Common mistake:** Returning `grad_out` unchanged for a bias vector.

7. Why can in-place mutation break autograd?

- **Answer:** It may modify a value saved for backward, invalidating the derivative; version counters detect many such cases.
- **Expected:** Declare mutations and prefer functional forms.
- **Common mistake:** Using `.data` or raw writes to bypass checks.

8. What does `gradcheck` do?

- **Answer:** It compares analytical gradients to finite-difference numerical estimates, usually using double precision.
- **Expected:** Smooth points/tolerances and separate registration testing.
- **Common mistake:** Running only float16 or at a non-differentiable point.

9. What is gradient accumulation?

- **Answer:** Contributions from all graph paths are summed, and leaf `.grad` buffers accumulate across backward calls unless reset.
- **Expected:** Explain why optimizers call `zero_grad`.
- **Common mistake:** Expecting `.backward()` to overwrite existing gradients.

10. How do you support double backward?

- **Answer:** Write the backward using differentiable, traceable PyTorch operations or provide further derivative support, then test with `gradgradcheck`/higher-order use.
- **Expected:** First-order support is not sufficient.
- **Common mistake:** Detaching values or using raw kernels inside backward and assuming second derivatives work.

7. Deep-Dive Questions

1. Why is reverse mode preferred for scalar-loss neural-network training?

- One reverse traversal computes derivatives of one/few outputs with respect to many inputs/parameters. Forward mode would need many directional passes to recover all parameter gradients.

2. What if backward needs a non-traceable native computation?

- Register that computation as a separate custom operator with its own metadata and derivative behavior as needed, then call it from the registered backward. This keeps the outer formula visible to transforms.

3. How do saved-tensor hooks or activation checkpointing relate to a custom op?

- Saved tensors participate in framework memory policies. Proper `save_for_backward` use lets hooks offload/compress state, while checkpointing trades recomputation for memory at a wider graph level.

4. Why can finite-difference gradcheck fail at a mathematically valid nonsmooth point?

- Operations such as absolute value, max ties, or clipping lack a unique derivative at some points. Small positive and negative perturbations observe different slopes, so choose random points away from discontinuities or document subgradient behavior.

5. How should complex-valued gradients be considered?

- PyTorch uses conjugate Wirtinger derivative conventions for real-valued losses. Backward formulas need correct conjugation; blindly copying a real-valued formula can be wrong.

8. Comparison Tables

Forward pass	Backward pass
Computes output values	Computes VJPs for inputs
May save minimal state	Consumes saved state and upstream grads
Uses CPU/CUDA kernel	Uses registered formula and possibly kernels
Runs in program order	Traverses graph in reverse dependency order
<code>torch.library.register_autograd</code>	<code>torch.autograd.Function</code>
Attaches formula to a registered operator	Defines a Python-level custom autograd node
Preferred for <code>torch.library</code> /C++ custom ops	Useful for custom differentiation workflows not already modeled as an op
Operator identity remains central	Called through <code>Function.apply</code>
Designed for operator subsystem composition	Requires care with newer transforms/compilers
<code>opcheck</code>	<code>gradcheck</code>
Tests registration contracts	Tests derivative mathematics
Checks schema/fake/autograd integration aspects	Compares analytical and numerical gradients
Representative subsystem inputs	Usually double/complex-double small inputs
Does not prove formula correctness	Does not replace schema/dispatch checks
Reverse mode	Forward mode
Vector-Jacobian product	Jacobian-vector product
Efficient for few outputs, many inputs	Efficient for few inputs/directions, many outputs
Standard neural-network backprop	Useful for directional derivatives/JVP workloads

9. Common Mistakes

- Implementing only forward and expecting training to work.
- Forgetting to multiply by `grad_out`.
- Returning gradients in the wrong input order.
- Returning the wrong number of gradient slots.
- Ignoring broadcast reduction to original input shapes.
- Saving more tensors than backward requires.
- Mutating saved tensors or bypassing version counters.
- Detaching inside backward and accidentally disabling higher-order gradients.
- Using `opcheck` as proof of gradient mathematics.
- Running `gradcheck` in low precision or at discontinuities.
- Expecting non-leaf `.grad` to be populated automatically.
- Forgetting that gradients accumulate across backward calls.

10. Edge Cases / Special Cases

- **Unused output gradient:** In multi-output ops, an unused output may deliver `None`; formulas must follow the API's materialization behavior.
- **Inputs not requiring grad:** `ctx.needs_input_grad` can skip expensive derivative work.
- **Broadcasting:** Reduce gradients to the exact original input sizes.
- **Views:** Gradients must respect alias/view semantics declared by the operator.
- **In-place forward:** Mutation and dirty-state semantics must be correct; functional designs are easier.
- **Integer/bool tensors:** They generally do not participate in ordinary floating/complex autograd.
- **Nondifferentiable points:** Specify subgradient behavior and choose gradcheck samples carefully.
- **Complex inputs:** Apply PyTorch's conjugation convention.
- **Sparse tensors:** Gradient layout may differ from dense assumptions; declare and test supported layouts.
- **Mixed precision:** Backward accumulation dtype and autocast policy affect stability.
- **Empty tensors:** Gradients should have correct empty shapes and device/dtype.
- **Multiple devices:** Saved tensors and grad outputs must be used on compatible devices and streams.
- **Non-deterministic forward:** Recomputing it in backward may not reproduce saved state.
- **Higher order:** An opaque backward kernel needs its own derivative support if double backward is promised.

11. How to Explain in Interview

"Autograd integration supplies the vector-Jacobian product for a custom operator. During forward I save only the tensors needed later. During backward I combine the upstream gradient with local derivatives and return one gradient slot per forward input, reducing broadcasted gradients to original shapes. For a registered op I attach the formula with `torch.library.register_autograd`, then use `opcheck` for integration and `gradcheck`—plus higher-order checks if required—for mathematical correctness."

12. Quick Revision Notes

- Backward computes VJPs, not usually a full Jacobian.
- Always include the upstream gradient in the chain rule.
- Save only what backward needs via `save_for_backward`.
- Return one slot per input; `None` for non-differentiable/non-tensor inputs.
- Reduce broadcasted gradients to original shapes.
- Gradients from multiple paths and backward calls accumulate.
- In-place mutation can invalidate saved state; version counters help detect it.
- Registered custom op: prefer `register_autograd`.
- `opcheck` checks integration; `gradcheck` checks derivative math.
- First-order support does not guarantee double backward.

13. Practice Tasks

1. Register the shown autograd formula for `mymuladd` and compare gradients with the pure PyTorch reference.
2. Run `gradcheck` on CPU double tensors, then on CUDA double tensors if supported.
3. Deliberately swap `grad_a` and `grad_b`; confirm gradcheck catches it.
4. Extend the operator to allow a broadcasted bias tensor and implement sum-to-size in backward.

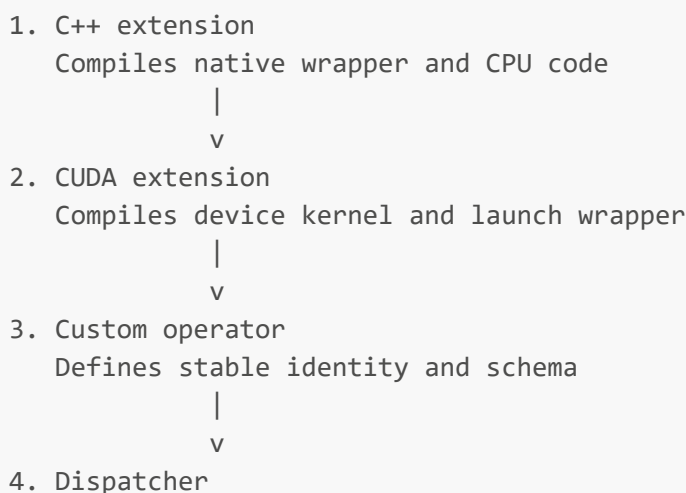
5. Create a two-output operator and return one gradient contribution per output/input path.
6. Test `requires_grad` on only `a`, only `b`, both, and neither.
7. Call `.backward()` twice and observe accumulation; then reset gradients correctly.
8. Run a higher-order derivative check and identify which operations in backward must remain differentiable.
9. Mutate a saved input after forward in a safe experiment and observe autograd's version-counter error.

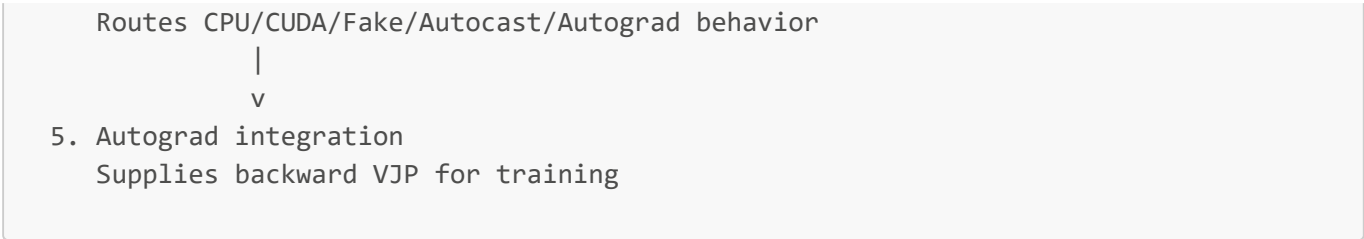
14. Final Cheat Sheet

Item	Must remember
Core definition	Derivative/VJP rule connecting a custom op to autograd
Forward responsibility	Compute output and save minimal required state
Backward responsibility	Apply chain rule to upstream grads
Running formula	<code>grad_a=g*b, grad_b=g*a</code> , scalar <code>c</code> -> <code>None</code>
Broadcast rule	Sum gradient back to original input shape
Mutation safety	Saved tensors must not be silently changed
Registered-op API	<code>torch.library.register_autograd</code>
Tests	Reference gradients, <code>gradcheck</code> , optional <code>gradgradcheck</code> , plus <code>opcheck</code>
Main trap	A correct forward kernel says nothing about gradient correctness
One-line answer	"Autograd integration registers a traceable VJP that maps output gradients to correctly shaped input gradients."

End-to-End Integration Blueprint

The five topics fit together as one pipeline:





Minimal file-level architecture

```
mylib_ops/  
├─ mymuladd.cpp           # schema + CPU implementation/registration  
├─ mymuladd_cuda.cu      # CUDA kernel + CUDA registration  
├─ registrations.py      # fake + autograd (+ autocast if needed)  
├─ setup.py              # CppExtension/CUDAExtension build  
└─ test_mymuladd.py      # reference, opcheck, gradcheck, edge cases
```

End-to-end verification checklist

Layer	Smallest convincing check
Forward semantics	Compare CPU and CUDA outputs with $a * b + c$
Boundary contract	Invalid shape/device/dtype fails clearly
Layout	Test contiguous and documented non-contiguous behavior
Dispatcher	CPU input reaches CPU kernel; CUDA reaches CUDA kernel
Fake/compile	<code>opcheck</code> passes on representative samples
Autograd	<code>gradcheck</code> passes with double inputs
Higher order	<code>gradgradcheck</code> if promised
Streams	Current-stream dependency test passes without device-wide sync
Performance	Warm CUDA-event benchmark against native reference

Official references

- [Custom C++ and CUDA Operators](#)
- [torch.library API](#)
- [Extending PyTorch](#)
- [C++ Operator Registration](#)